Genesis Alarm Forwarding Configuration

Table of contents

- Introduction
- Architecture Overview
- Primary CMS System
  - **Overview**
  - Supported Primary CMS Systems
    - **When to Use This**
- Traditional / Industry Standard CMS Systems
  - Overview
  - Supported Traditional CMS Systems
    - **When to Use This**
- Technological Alarm Forwarders
  - Overview
  - Supported Technological Forwarders
    - When to Use This
  - Configuration - Alarm Forwarder
  - Webhook Forwarder Configuration
    - **Configuration Breakdown**
    -
  - File Transfer Forwarder Configuration
    - Configuration Breakdown
  - Email Forwarder Configuration
    - Configuration Breakdown

- MQTT Forwarder Configuration
  - Configuration Breakdown
- How the Categories Work Together
  - Use Case Summary
  - Detailed Use Cases
  - **1. Existing Evalink Talos Customer (Cloud-Only Customer)**
    - **Who is this for?**
    - **Use Case Description**
    - **Typical Customer Example**
  - **2. Customer Migrating from AmWin/ATS/Legacy CMS to Talos (Legacy CMS Migration + Cloud Future)**
    - **Who is this for?**
    - **Use Case Description**
    - **Typical Customer Example**
  - **3. Hybrid Monitoring Setup (Talos + IMMIX + Technological Services)**
    - **Who is this for?**
    - **Use Case Description**
    - **Typical Customer Example**
  - **4. Highly Customized / Tech-Driven Monitoring Stations (Build-their-own CMS)**
    - **Who is this for?**
    - **Use Case Description**
    - **Typical Customer Example**

Introduction

Genesis provides a unified, cloud-native platform for receiving, enriching, and forwarding alarms to any monitoring backend.
Because different customers operate in different technological maturities: modern cloud CMS, legacy DC-09 receivers, hybrid infrastructures, or custom-built monitoring stacks. Genesis supports **three categories of alarm forwarders**:

1. **Primary CMS Forwarders** – A tightly integrated and certified connection with leading cloud CMS platforms. Currently, Genesis supports **Evalink Talos** as a native primary CMS.

2. **Traditional CMS Forwarders** – For customers using industry-standard or legacy receivers such as **IMMIX**, **ATS, Sur-Gard**, **AmWin**, **LISA**, and similar platforms.

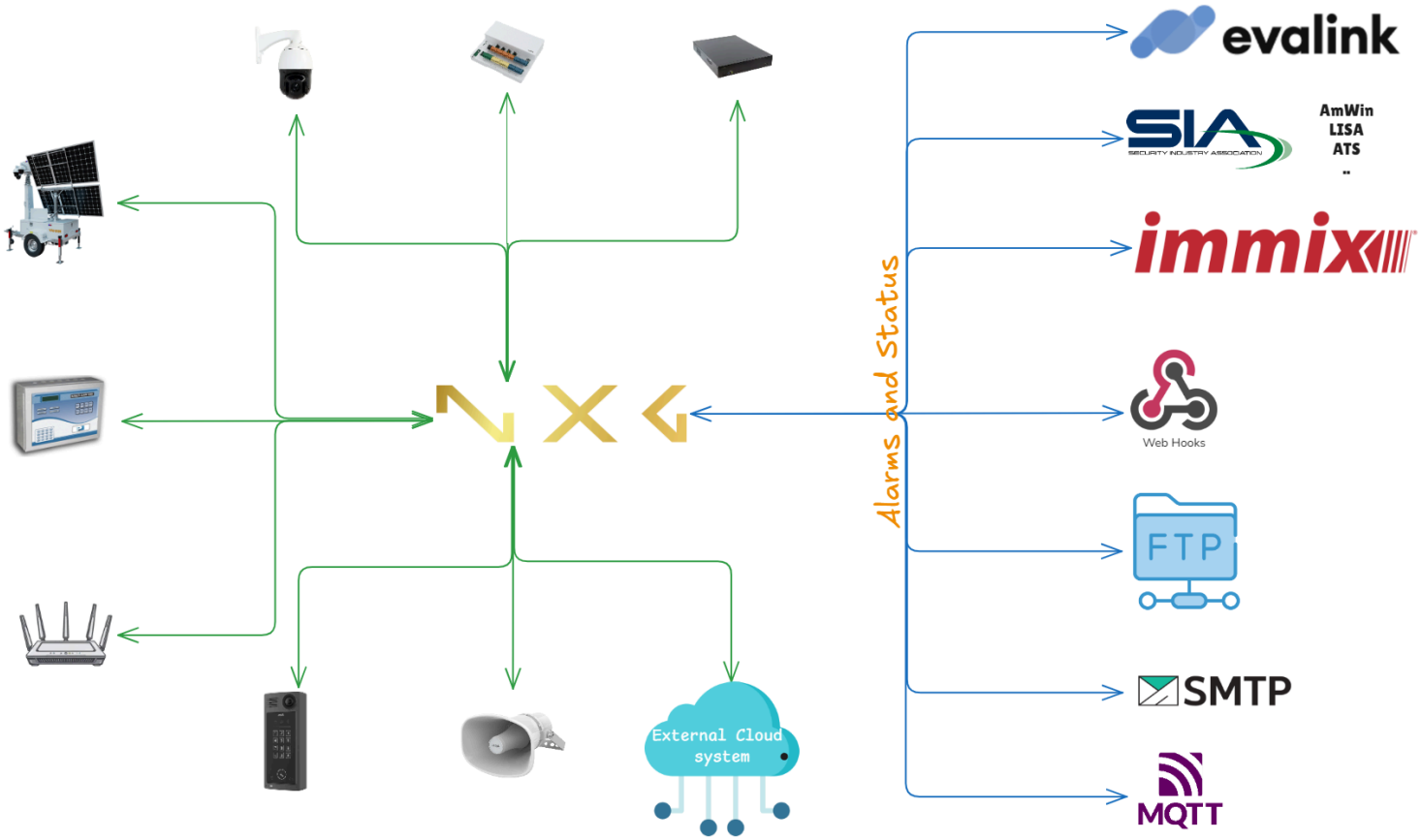
3. **Technological Forwarders** – For modern, API-driven monitoring models using **Webhooks, Email, File Transfer**, or **MQTT**.

Each category enables a customer to route alarms according to their operational model: cloud-only, migration from legacy, hybrid use, or fully customized workflows.

Genesis allows customers to enable **one option per category**, but they may combine categories depending on their monitoring strategy. For example: Talos (Primary) + AmWin (Traditional) + Webhook (Technological).

Architecture Overview

Genesis is designed as a **modular alarm-processing pipeline**, where device events pass through standardized internal layers before being forwarded to the customer's selected monitoring platform(s).

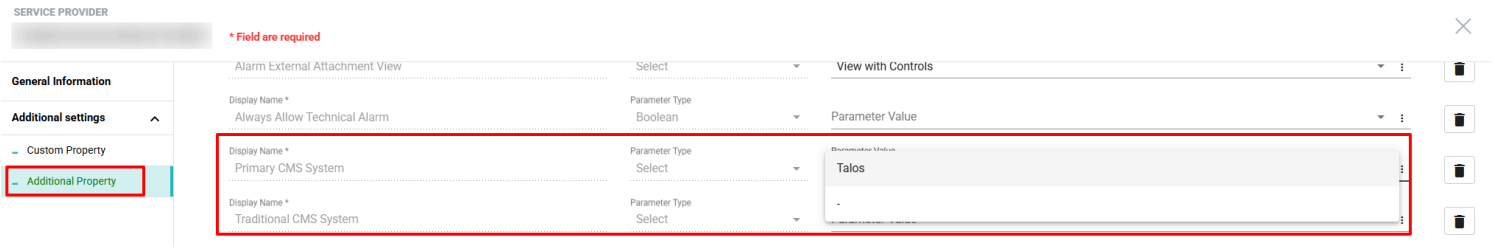


Primary CMS System

Deep, system-level integration with a next-generation cloud-native CMS

Overview

The Primary CMS System is the *preferred* alarm forwarding pipeline for customers who require **real-time**, **bi-directional**, and **high-reliability** integration with an advanced cloud CMS.
This is *not* a generic protocol-based forwarder—it is a **tightly coupled integration** that synchronizes events, sites, assets, health, video clips, metadata, enriched AI signals, and operator workflows.



Supported Primary CMS Systems

CMS	Integration Type	Supported by Genesis
Evalink Talos	Full cloud-native API integration	Supported

When to Use This

Use the **Primary CMS System (Evalink Talos)** integration in the following situations:

1. Customer already uses Evalink Talos as their main monitoring backend

- Genesis becomes an extension of their existing Evalink environment.
- All alarms, AI events, device health, and site details synchronize automatically from Genesis to Talos.
- Operators work entirely inside Evalink with Genesis acting as the field intelligence layer.

2. Customer wants to use Genesis *together* with Evalink, even if they do not have a standalone Evalink deployment

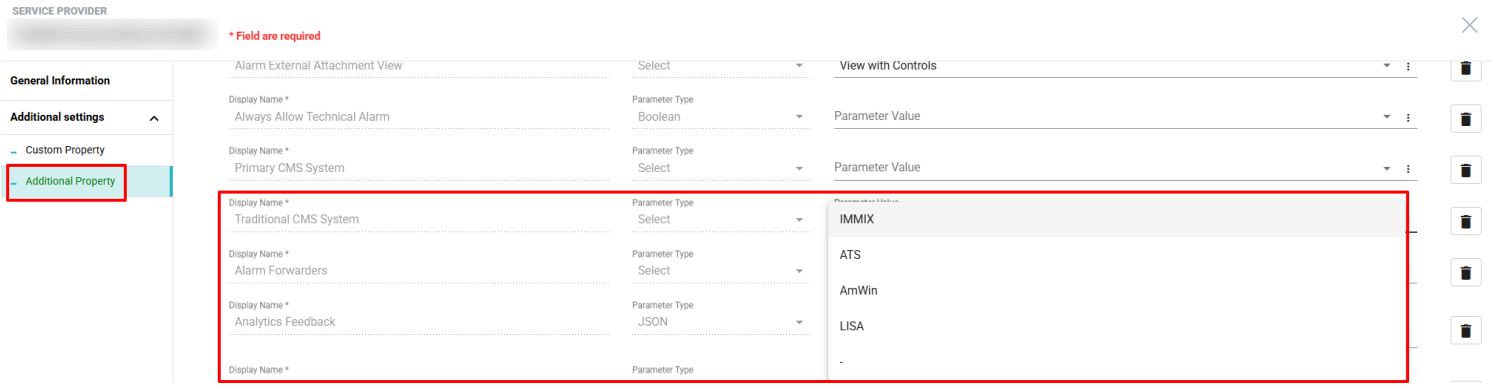
- Some customers prefer Evalink's operator console and workflow but rely on Genesis for tower/device management.
- In this model, Evalink becomes the CMS front-end and Genesis system becomes the IoT/Video/AI processing engine.
- Genesis provisions Evalink accounts, pushes events, and synchronizes sites on behalf of the customer.
- No separate Evalink system is required—the integration makes the Genesis + Evalink experience seamless.

Traditional / Industry Standard CMS Systems

Protocol-based forwarding for legacy or industry-standard Central Monitoring Stations

Overview

These integrations are for customers who use traditional CMS systems that operate on standard alarm receiver protocols (e.g., Sur-Gard, SIA DC-09, or custom vendor-specific variations).
These platforms do **not** have cloud-native APIs; they rely on persistent sockets, framed messages, and legacy alarm codes.



Supported Traditional CMS Systems

CMS	Protocol	Direction	Supported by Genesis
IMMIX	SMTP (email receiver)	Genesis to CMS	Supported
ATS	SIA DC-09	Genesis to CMS	Supported
AmWin	SIA DC-09	Genesis to CMS	Supported
LISA	Custom SIA D-09 variant	Genesis to CMS	Supported
Sur-Guard	Sur-Guard	Genesis to CMS	Not Supported

When to Use This

Use the **Traditional/Industry-Standard CMS Forwarders** when your monitoring workflow depends on established alarm-receiver formats or legacy integration methods.

1. When your organization uses a traditional national or regional CMS infrastructure

Many national CMS operators still rely on hardware alarm receivers or long-established protocols.
This option ensures Genesis events integrate seamlessly with those environments.

2. When you need compatibility with existing CMS platforms (IMMIX, ATS, AMWIN, LISA, etc.)

Choose this category when your backend requires:

- **SMTP email ingestion** (e.g., IMMIX)
- **DC-09 alarm frames** (e.g., ATS D09, AMWIN DC09, LISA custom D09)

3. When migrating gradually from legacy CMS to modern cloud architecture

This option supports hybrid environments where:

- The customer continues using legacy CMS receivers
- Genesis delivers alarms in backward-compatible formats
- Migration can happen at their own pace without interruption

Genesis normalizes all events and converts them into protocol-compliant frames expected by each CMS.

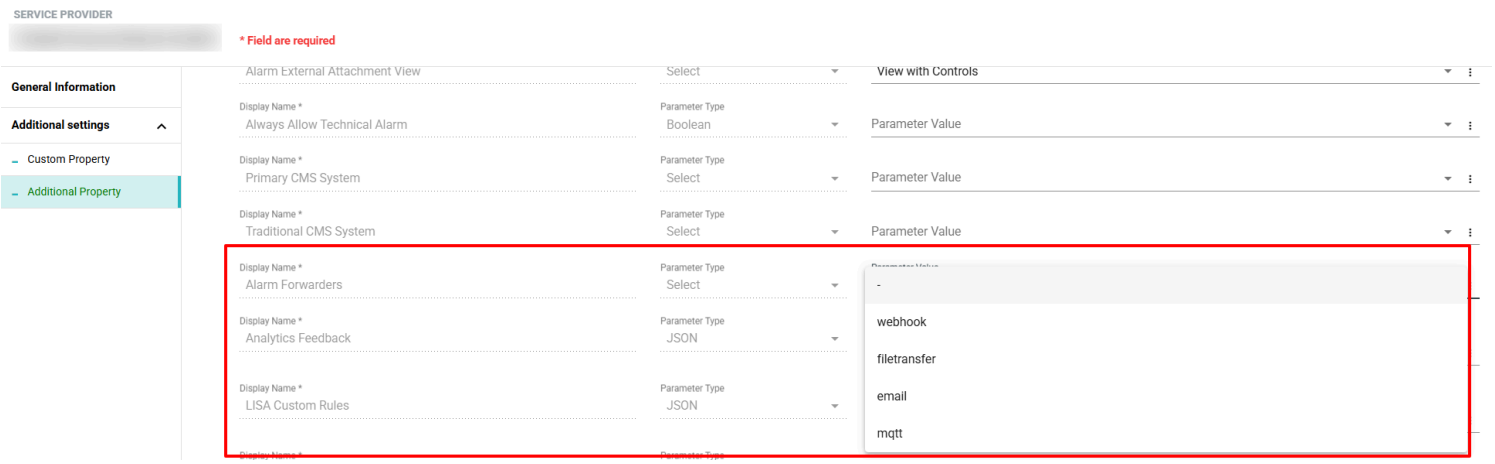
Technological Alarm Forwarders

Modern, flexible, API-friendly distribution channels

Overview

These integrations are designed for customers who want to:

- Organizations building their own CMS or working with technology partners who use customized monitoring platforms.
- Feed alarms into automation platforms (Node-RED, Azure Logic Apps, n8n, Zapier).
- Store alarms in external systems or data lakes.
- Integrate non-CMS business applications.



Supported Technological Forwarders

Forwarder	Transport	Direction	Supported by Genesis
Webhook	HTTPS POST	Genesis to Customer API Server	Work in Progress
File Transfer	FTP/SFTP	Genesis to Customer FTP server	Work in Progress
Email	SMTP	Genesis to Customer Email Server	Work in Progress
MQTT	MQTT Broker	Genesis to Customer MQTT Broker	Not Supported

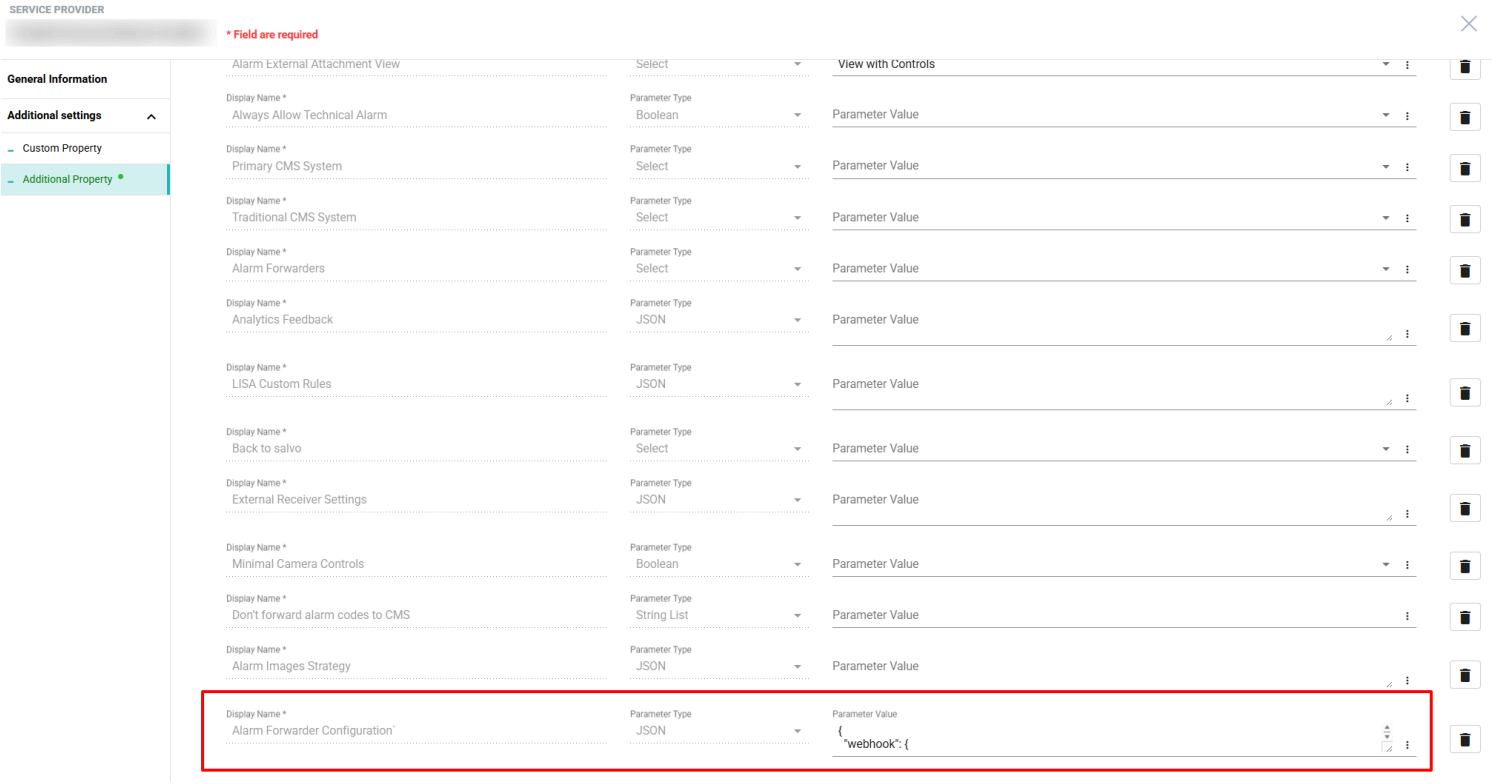
When to Use This

- To integrate with automation or orchestration tools
- For serverless or custom cloud workflows
- For IoT or message-driven systems
- When CMS functionality is not required but alarm notifications are needed

Configuration - Alarm Forwarder

The Genesis platform allows customers to route alarms and events to different backend systems through a flexible, JSON-based configuration.
Each forwarder type—**Webhook**, **File Transfer**, **Email (SMTP)**, and **MQTT**—supports its own configuration block with customizable properties and retry policies. It can be configurable in Service Provider/Customer/Site/Device level.

This section explains each field in detail so customers can configure their integrations safely and correctly.



Webhook Forwarder Configuration

Webhooks enable real-time delivery of alarms to any HTTP/HTTPS endpoint.

```
"webhook": {
  "type": "webhook",
  "properties": {
    "url": "https://customer-endpoint.com/alarms",
    "method": "POST",
    "auth": {
      "type": "none | basic | digest | bearer | apiKey | custom",
      "basic": { "username": "", "password": "" },
      "digest": { "username": "", "password": "" },
      "bearer": { "token": "" },
      "apiKey": {
        "location": "header | query",
        "keyName": "X-API-Key",
        "keyValue": "123456"
      },
      "customHeaders": {
```

```
      "X-Custom-1": "value1",
      "X-Custom-2": "value2"
    }
  },
  "payloadTemplate": "{ \"event\": \"{{eventType}}\", \"timestamp\": \"{{timestamp}}\" }",
  "attachment": {
    "enabled": false,
    "fileFormat": "mp4",
    "filenameTemplate": "{{siteName}}_{{deviceName}}_{{cameraName}}_{{eventType}}"
  }
 },
 "retry_policy": {
  "max_retries": 3,
  "retry_interval_sec": 10
 }
}
```

Configuration Breakdown

Name	Description
url	**Destination endpoint URL where alarms are delivered.**
method	**HTTP method used. Supported: POST**
auth	**Authentication Options:** **none - No authentication.** **basic - Standard username/password Base64 auth.** **digest - Digest challenge-response used by some legacy systems.** **bearer - Token-based authorization (e.g., JWT, Auth0 static tokens, etc..)** **apiKey - API Key sent via header or query parameter.** **custom - Allows sending arbitrary custom headers.**
payloadTemplate	**A templated JSON body. Genesis will replace variables like {{eventType}}, {{timestamp}}, {{siteName}}, etc.** **Example: payloadTemplate: "{ \"event\": \"{{eventType}}\", \"timestamp\": \" {{timestamp}}\" }"**
Attachment Options	**enabled - Whether to attach media (snapshot/video).** **fileFormat - mp4, jpeg.** **filenameTemplate - Dynamic naming based on event metadata.**

Retry Policy	**If the webhook server fails (5xx, timeout), Genesis retries:** **max_retries - Number of attempts before failure.** **retry_interval_sec - Delay between attempts.**

File Transfer Forwarder Configuration

Used when a CMS or backend system ingests files via FTP/FTPS.

```
"filetransfer": {
  "type": "file_transfer",
  "properties": {
    "protocol": "ftp | ftps",
    "host": "file.customer.com",
    "port": 21,
    "username": "alarm_user",
    "password": "xxxx",
    "secure": false,
    "path": "/incoming/alarms",
    "fileFormat": "mp4",
    "filenameTemplate": "alarm-{{eventId}}"
  },
  "retry_policy": {
    "max_retries": 5,
    "retry_interval_sec": 30
  }
}
```

Configuration Breakdown

Name	Description
protocol	• **ftp** - Standard FTP • **ftsp** - FTP with TLS
secure	If `true`, forces FTPS explicit TLS mode.
path	Directory where files are uploaded.

File Output	• **fileFormat** - mp4, jpg. • **filenameTemplate** - Template-driven dynamic filenames.

Email Forwarder Configuration

Sends alarms to email recipients, often used for older monitoring systems such as IMMIX.

```
"email": {
  "type": "smtp",
  "properties": {
    "to": ["ops-team@customer.com"],
    "subjectTemplate": "Alarm - {{eventType}} at {{siteName}}",
    "bodyTemplate": "Alarm: {{eventType}} occurred at {{timestamp}}.",
    "attachment": {
      "enabled": false,
      "fileFormat": "jpg",
      "filenameTemplate": "{{eventId}}_snapshot"
    }
  },
  "retry_policy": {
    "max_retries": 5,
    "retry_interval_sec": 60
  }
}
```

Configuration Breakdown

Name	Description
to	Recipients
Templates	Support templating for full customization. subjectTemplate, bodyTemplate
Attachment block	Same structure as Webhook forwarder.

MQTT Forwarder Configuration

Used for IoT platforms, real-time event buses, and custom CMS systems.

```
"mqtt": {
  "type": "mqtt",
  "properties": {
    "brokerUrl": "mqtt://broker.customer.com:1883",
    "username": "mqtt_user",
    "password": "xxxx",
    "topic": "alarms/{{siteId}}",
    "qos": 1,
    "retain": false,
    "payloadTemplate": "{ \"event\": \"{{eventType}}\", \"timestamp\": \"{{timestamp}}\", \"device\": \"{{deviceName}}\" }"
  },
  "retry_policy": {
    "max_retries": 10,
    "retry_interval_sec": 5
  }
}
```

Configuration Breakdown

Name	Description
brokerUrl	mqtt:// and ws:// (MQTT over websockets) based broker url
topic	Fully template-based topic routing, e.g.: alarms/tower-{{siteId}}/device-{{deviceId}}
QoS Levels	0 - fire and forget, 1 - Atleast Once, 2 - Exactly Once
Payload Template	JSON payloads with dynamic fields.

How the Categories Work Together

Customer may enable up to 3 forwarders at the same time

- **0 or 1 Primary CMS** (Talos or None)
- **0 or 1 Traditional CMS** (IMMIX / ATS / AmWin / LISA / None)
- **0 or 1 Technological Forwarder** (Webhook / File / Email / MQTT / None)

Use Case Summary

Scenario	Primary	Traditional	Technological
Cloud only Customer	Talos	None	None
Legacy CMS Integration	None	ATS	None
Hybrid	Talos	AmWin	Email
Highly Customized	None	None	Webhook

Detailed Use Cases

Below are the **derived explanations** of what each scenario represents in real customer deployments.

1. Existing Evalink Talos Customer (Cloud-Only Customer)

Primary = Talos | Traditional = None | Technological = Webhook

Who is this for?

Customers who already run their monitoring operations on **Evalink Talos** and want Genesis to seamlessly feed alarms, AI events, and device health to their Talos environment.

Use Case Description

- Customer operates fully in the cloud and does not use any legacy receivers or DC-09 infrastructure.
- Genesis acts as the tower/device/AI processing layer, while Talos remains the operator console and workflow engine.
- Optional: Technological forwarders (Webhook, API, MQTT) may be used for analytics, automation, or forwarding events to third-party tools.

Typical Customer Example

A global cloud-first monitoring company using Talos for all operator workflows, wanting Genesis to integrate video alarms, AI metadata, and tower telemetry directly into Talos in real time.

2. Customer Migrating from AmWin/ATS/Legacy CMS to Talos (Legacy CMS Migration + Cloud Future)

Primary = None → later Talos | Traditional = ATS/AmWin | Technological = None

Who is this for?

Organizations using legacy CMS systems such as ATS or AmWin and slowly transitioning to cloud-native solutions like Evalink Talos.

Use Case Description

- Customer still relies on traditional DC-09 receivers for their current alarm workflow.
- Genesis sends alarms in **ATS D09**, **AmWin DC-09**, or **custom D09** formats during the transition.
- Once migration completes, the customer switches **Primary = Talos**.

Typical Customer Example

A nationwide CMS using ATS hardware receivers migrating their sites in phases to Talos.
During transition:

- Traditional alarms → ATS receiver
- AI events + IoT telemetry → Talos and validate in parallel.
- Future: Everything consolidated into Talos and turn off ATS.

3. Hybrid Monitoring Setup (Talos + IMMIX + Technological Services)

Primary = Talos | Traditional = IMMIX | Technological = Email or Webhook

Who is this for?

Large enterprises or bureau operators running **multiple CMS platforms in parallel**.

Use Case Description

- Evalink Talos handles advanced cloud monitoring, collaboration, and workflow automation.
- IMMIX is still used for a part of their operations (e.g., remote guarding workflows).
- Genesis feeds both systems simultaneously:
    - Talos: AI alarms, device health, tower telemetry (Primary)
    - IMMIX: Camera-based alarms via SMTP/Email
    - Technological: Extra routing via Email/Webhook for internal tools or automation

Typical Customer Example

A security-as-a-service provider with:

- Talos for national operations center
- IMMIX for guard-force integration and dispatch workflows
- Email/Webhook for internal automation and reporting

This hybrid approach is common for customers with diverse operational units or multiple subcontractors.

4. Highly Customized / Tech-Driven Monitoring Stations (Build-their-own CMS)

Primary = None | Traditional = None | Technological = Webhook/MQTT/File Transfer

Who is this for?

Organizations building their own monitoring backend or using a custom technology partner—not relying on any off-the-shelf CMS.

Use Case Description

- Customer builds a proprietary CMS or automation engine.
- Genesis provides raw or enriched event streams via:
    - **Webhooks** for real-time event delivery
    - **MQTT** for IoT-style message pipelines
    - **SFTP/FTP** for batch processing
    - **Email** for alerting fallback
- Customer has full control over how they ingest, enrich, or process alarms.

Typical Customer Example

A technology-driven monitoring provider building a fully custom cloud-native CMS using:

- Webhooks to receive alarms
- Their own AI engines to reprocess events
- Custom dashboards for operators
- No traditional CMS receiver at all

This is also common for:

- Smart city platforms
- Autonomous surveillance towers
- Industrial automation integrators
- Border/perimeter monitoring with custom workflows